

Ejemplos de replicación y sharding con MongoDB

En esta práctica vamos a ver como montar un cluster de varias máquinas con MongoDB, para dar soporte a la replicación y sharding de las colecciones de documentos. Simularemos las máquinas del cluster con varios contenedores docker.

Ejemplo de creación de un cluster con replicación en MongoDB

En el tutorial del manual de MongoDB hay secciones independientes para desplegar un [cluster de pruebas](#), como el que vamos a ver aquí, y un [cluster de producción](#).

Aquí nos centraremos en un cluster de pruebas o desarrollo, para facilitar este ejemplo, y vamos a utilizar solo una máquina en la que levantaremos tres instancias distintas (procesos) de MongoDB, que simularán tres nodos del cluster.

Cuando ejecutamos el contenedor docker mongo:latest, automáticamente, se ejecuta dentro el comando mongod para levantar una instancia de mongod. Para tener más control sobre la ejecución de este proceso, vamos a reemplazar el comando mongod por un comando bash que nos ejecute una terminal. Después desde la terminal ejecutamos los comandos y procesos necesarios. Además, necesitamos que los contenedores se comuniquen entre sí, por lo que debemos crear una red también.

OJO: Si paras y vuelves a arrancar el contenedor, tendrás que volver a levantar los procesos para que el cluster funcione de nuevo. En un entorno real esto no se haría así. Para ver cómo desplegar un cluster en producción, puedes consultar el manual de MongoDB.

Primero creamos la red (si ya tenemos la red creada podemos saltarnos este paso)

```
docker network create network-xine-grei
```

Ahora creamos el contenedor del primer nodo y abrimos una terminal para levantar el proceso

?

```
docker run --name xine-grei-mongodb-rs0-1 -dti -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei
mongo:latest bash

docker exec -it xine-grei-mongodb-rs0-1 bash
```

Creamos una carpeta para almacenar los datos y levantamos el proceso

```
mkdir /home/alumnobd/mongo

nohup mongod --replSet rs0 --port 27017 --bind_ip_all --dbpath /home/alumnobd/mongo > /dev/null &
```

La opción `--replSet` se utiliza para darle un identificador al replicaset. Debemos usar el mismo identificador en los tres nodos. La opción `--bind_ip_all` permite recibir peticiones desde cualquier ip. La opción `--dbpath` indica la ruta de la carpeta donde se van a guardar los archivos de datos de la instancia. Nohup permite que el proceso siga ejecutándose aunque cerremos la terminal. Redireccionamos la salida a `/dev/null` para no tener mensajes de log en la terminal.

Con "ps -xa" puedes comprobar que el proceso está funcionando.

Repetimos los pasos anteriores para los otros dos contenedores del cluster.

```
docker run --name xine-grei-mongodb-rs0-2 -dti -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei
mongo:latest bash

docker exec -it xine-grei-mongodb-rs0-2 bash

mkdir /home/alumnobd/mongo
nohup mongod --replSet rs0 --port 27017 --bind_ip_all --dbpath /home/alumnobd/mongo > /dev/null &

docker run --name xine-grei-mongodb-rs0-3 -dti -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei
mongo:latest bash

docker exec -it xine-grei-mongodb-rs0-3 bash

mkdir /home/alumnobd/mongo
nohup mongod --replSet rs0 --port 27017 --bind_ip_all --dbpath /home/alumnobd/mongo > /dev/null &
```

Ahora debemos iniciar el replicaset ejecutando un comando en cualquiera de las tres instancias. Vamos a usar la instancia rs0-1.

```
docker exec -it xine-grei-mongodb-rs0-1 bash
mongosh
```

El comando a ejecutar es el siguiente.

```
rs.initiate(
{
  _id: "rs0",
  members: [
    {
      _id: 0,
      host: "xine-grei-mongodb-rs0-1:27017"
    },
    {
      _id: 1,
      host: "xine-grei-mongodb-rs0-2:27017"
    },
    {
      _id: 2,
      host: "xine-grei-mongodb-rs0-3:27017"
    }
  ]
}
)
```

En cada miembro del replicaset estamos indicando un identificador para el miembro (0, 1, 2), y una referencia al proceso, indicando la máquina y el puerto. Para poder conectar con el replicaset usamos, desde cualquiera de los tres contenedores, el siguiente comando.

```
mongosh "mongodb://xine-grei-mongodb-rs0-1:27017,xine-grei-mongodb-rs0-2:27017,xine-grei-mongodb-rs0-3:27017/?
replicaSet=rs0"
```

Con los siguientes comandos podemos ver información sobre la configuración y el estado del replicaset.

```
rs.conf()  
rs.status()
```

Vamos a crear una colección de prueba y verificar que podemos consultar sus datos incluso si alguno de los nodos deja de funcionar.

```
use xine  
  
db.coleccionPrueba.insertOne({clave:1, valor:5})  
  
for (var i=2; i<1000; i++) {db.coleccionPrueba.insertOne({"clave":i,"valor":i+5})}  
  
db.coleccionPrueba.find().count()
```

Ahora pausamos uno de los contenedores (que no sea aquel en el que estamos ejecutando los comandos). Probamos de nuevo la consulta y comprobamos que se ejecuta sin problema.

```
db.coleccionPrueba.find().count()
```

Pausamos ahora otro contenedor más y comprobamos de nuevo si podemos leer (Asegúrate de pausar y no parar, para que no tengas que volver a ejecutar los procesos cuando vuelvas a iniciar). Si esperamos el tiempo suficiente el nodo puede cambiar de nodo primario a secundario y veremos que no se puede leer a menos que se cambie la preferencia de lectura de "primary" a "primaryPreferred".

```
db.getMongo().setReadPref("primaryPreferred")
```

Si volvemos a arrancar los contenedores veremos que ya todo funciona de nuevo. Podemos volver a poner la preferencia de lectura en su opción por defecto

```
db.getMongo().setReadPref("primary")
```

Una vez terminado el ejemplo puedes parar y eliminar los tres contenedores.

Ejemplo de creación de un cluster con sharding en MongoDB

En el manual tenemos un [tutorial](#) para desplegar un cluster con sharding, tanto usando directamente la línea de comando para iniciar los nodos, como usando archivos de configuración. Para simplificar, aquí vamos a usar una sola máquina y no vamos a usar replicación en los componentes.

Para desplegar nuestro cluster, y dado que no vamos a replicar ningún componente, necesitaremos iniciar los siguientes procesos:

- Un proceso mongod para cada uno de los dos shards que vamos a desplegar
- Un proceso mongod para el servidor de configuración del cluster
- Un proceso mongos para el broker que da acceso al cluster

Recordar que cada uno de los procesos de arriba, en un cluster de producción, debería de estar replicado como mínimo 3 veces, es decir, necesitaríamos 12 procesos.

Para simplificar este ejemplo, vamos a usar replicasets de un solo proceso para crear un cluster de dos shards. A nivel de arquitectura, vamos a utilizar tres contenedores para simular tres máquinas. En una máquina tendremos alojado el broker y la configuración y en las otras dos tendremos un shard en cada una.

Despliegue del servidor de configuración (Config Server)

Lo primero que vamos a hacer es desplegar el servidor de configuración en un contenedor que llamaremos broker. Para este contenedor, que es el punto de entrada al cluster, vamos a mapear un puerto para que pueda ser accedido desde el host si es necesario (por ejemplo para conectar con el cluster usando Compass).

Con los siguientes comandos, creamos el contenedor, abrimos una terminal, creamos una carpeta para los datos de configuración e iniciamos el replicaset de configuración.

```
docker run --name xine-grei-mongodb-broker -dti -p 27017:27017 -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei mongo:latest bash
docker exec -it xine-grei-mongodb-broker bash
mkdir /home/alumnobd/servconf
nohup mongod --configsvr --replSet rsCS --port 27018 --bind_ip_all --dbpath /home/alumnobd/servconf > /dev/null &
ps -xa
mongosh --port 27018

rs.initiate({
  _id: "rsCS",
  configsvr: true,
  members: [{_id: 0, host: "xine-grei-mongodb-broker:27018"}]
})
```

Creación de los replicaSet de los dos shards

Ahora creamos dos contenedores más, uno para cada uno de los shards.

```
docker run --name xine-grei-mongodb-shard1 -dti -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei
mongo:latest bash
docker exec -it xine-grei-mongodb-shard1 bash
mkdir /home/alumnobd/shard1
nohup mongod --shardsvr --replSet rsSH1 --port 27017 --dbpath /home/alumnobd/shard1 --bind_ip_all > /dev/null &
ps -xa
mongosh --port 27017

rs.initiate({
  _id: "rsSH1",
members: [{_id: 0, host: "xine-grei-mongodb-shard1:27017"}]
})

docker run --name xine-grei-mongodb-shard2 -dti -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei
mongo:latest bash
docker exec -it xine-grei-mongodb-shard2 bash
mkdir /home/alumnobd/shard2
nohup mongod --shardsvr --replSet rsSH2 --port 27017 --dbpath /home/alumnobd/shard2 --bind_ip_all > /dev/null &
ps -xa
mongosh --port 27017

rs.initiate({
  _id: "rsSH2",
members: [{_id: 0, host: "xine-grei-mongodb-shard2:27017"}]
})
```

Despliegue del broker de consultas mongos y configuración del cluster

Primero iniciamos el proceso mongos para que actúe como punto de entrada al cluster.

```
docker exec -it xine-grei-mongodb-broker bash  
  
nohup mongos --port 27017 --configdb rsCS/xine-grei-mongodb-broker:27018 --bind_ip_all > /dev/null &
```

De usar varias instancias para cada replicaSet, la opción `--configdb` tendría la forma `"rsConfServer/ip1:port1,ip2:port2,ip3:port3"`.

Ahora ya podemos añadir los dos shards al cluster. Para ello tenemos que conectar con el broker y utilizar las funciones que permiten añadir nuevos replicaset como nuevos shards. Abrimos por tanto una terminal en el contenedor broker y ejecutamos los siguientes comandos:

```
docker exec -it xine-grei-mongodb-broker bash  
mongosh  
sh.addShard("rsSH1/xine-grei-mongodb-shard1:27017")  
sh.addShard("rsSH2/xine-grei-mongodb-shard2:27017")
```

Si se usan varias instancias en cada replicaSet, el formato para añadir cada shard sería `"rsShardid/ip1:port1,ip2:port2,ip3:port3"`.

Ahora vamos a crear una base de datos de prueba, habilitar el sharding en ella e insertar algunos datos para comprobar si se están repartiendo entre los shards o no.

```
use xine  
  
sh.enableSharding("xine")
```

Por defecto, no hay un valor especificado para el tamaño del *chunk*, y el sistema usa el valor de 128MB. Para cambiar el tamaño del *chunk* 1MB tenemos que añadir o modificar el campo "chunksize" de la colección "settings". Si no reducimos el tamaño de chunk, tendremos que insertar muchos datos antes de poder ver movimiento entre los shards.

```
use config  
db.settings.updateOne({ _id: "chunksize" }, { $set: { _id: "chunksize", value: 1 } }, {upsert:true})
```

Ahora podemos crear una colección nueva, indexarla (utilizaremos hashing) y particionarla.

```
use xine

db.coleccionPrueba.insertOne({clave:1, valor:"Un valor de texto cualquiera"})

db.coleccionPrueba.createIndex({clave:"hashed"})
```

Ahora particionamos la colección:

```
sh.shardCollection("xine.coleccionPrueba", {clave:"hashed"})
```

De momento, todos los datos estarán en un único shard, ya que todos caben en un único chunk de 1MB.

```
sh.status()
```

Ahora añadimos suficientes datos para que se generen más *chunks*. Puede tardar unos minutos en insertar todos los documentos.

```
for (var i=2; i<100000; i++) {db.coleccionPrueba.insertOne({"clave":i, "valor":"Otro valor de texto cualquiera, lo  
suficientemente largo para que el tamaño final aumente bastante"})}
```

Comprobamos finalmente cómo los chunks se repartieron entre los shards:

```
sh.status()
```

Debería haber unos 3 chunks, repartidos entre los dos shards.

Dado que no usamos replicación, podemos comprobar que al detener cualquiera de los tres componentes, el servidor de configuración o cualquiera de los dos *shards*, ya no podremos acceder a la colección de prueba que acabamos de crear.

Una vez que terminamos con el ejemplo ya podemos eliminar los contenedores.

Última modificación: luns, 9 de febreiro de 2026, 19:26